



Taylor & Francis
Taylor & Francis Group

Field Experiences with eXtreme Programming: Developing an Emergency Response System

Author(s): Ann Fruhling and Gert-Jan de Vreede

Source: *Journal of Management Information Systems*, Vol. 22, No. 4, Crossing Boundaries in Information Systems Research (Spring, 2006), pp. 39-68

Published by: Taylor & Francis, Ltd.

Stable URL: <http://www.jstor.org/stable/40398813>

Accessed: 21-11-2016 07:53 UTC

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <http://about.jstor.org/terms>



Taylor & Francis, Ltd. is collaborating with JSTOR to digitize, preserve and extend access to *Journal of Management Information Systems*

Field Experiences with eXtreme Programming: Developing an Emergency Response System

ANN FRUHLING AND GERT-JAN DE VREEDE

ANN FRUHLING is an Assistant Professor in the Department of Information Systems and Quantitative Analysis, College of Information Science and Technology at the University of Nebraska at Omaha. Dr. Fruhling received her Ph.D. from the University of Nebraska at Lincoln. Her research includes agile methods including eXtreme Programming, human-computer interaction, usability, user interface design, e-health trust issues, and software engineering solutions for biosecurity decision support applications. Dr. Fruhling is involved in an extensive research project designing and implementing a public health emergency response system. She has published papers on usability user interface evaluation, eXtreme Programming, collaboration engineering, and emergency response systems, and is a recipient of several grants, including the Nebraska Research Initiative, Health and Human Services Regulation, and the Association of Public Health Laboratories.

GERT-JAN DE VREEDE is Kayser Professor at the Department of Information Systems and Quantitative Analysis at the University of Nebraska at Omaha where he is director of the Peter Kiewit Institute's Program on E-Collaboration. He is also affiliated with the Faculty of Technology, Policy and Management of Delft University of Technology in the Netherlands from where he received his Ph.D. His research focuses on field applications of e-collaboration technologies, the theoretical foundations of (e-)collaboration, the (un)successful implementation of e-collaboration technologies, the development of practitioner-driven collaborative processes, facilitation of group meetings, and the diffusion of collaboration technology. His articles have appeared in journals such as *Journal of Management Information Systems*, *Communications of the ACM*, *DataBase*, *Group Decision and Negotiation*, *Journal of Decision Systems*, *Journal of Creativity and Innovation Management*, *International Journal of Technology and Management*, *Journal of Informatics Education and Research*, *Simulation & Gaming*, *Simulation*, and *Journal of Simulation Practice and Theory*.

ABSTRACT: eXtreme Programming (XP) is a well-known agile software development method. While a number of reports have been published on explaining the XP methodology and the perceived benefits when using XP for system development in recent years, less is known about the actual operationalization of the XP principles. This paper presents an action research study reporting on the experiences of implementing the XP methodology in a development project for a Web-based, distributed information system. The goal of this research was to increase the understanding of how to effectively operationalize XP techniques so that the system being developed catered to today's fast-paced technological environment by allowing the developers to respond quickly to innovative and changing requirements.

Overall, the research indicates that most of the XP principles could be effectively implemented; however, three of the principles required modification (i.e., testing, pair programming, customer collocation). Several benefits resulted from the usage of XP. The rapid prototyping enabled information technology developers and users to clarify system requirements, communicate openly, quickly build rapport, and create an interface that was easy to use and learn. Further, the research found that where the technology was new or foreign to the development team and the user, the XP process was flexible enough to support several iterations of technology and produce prototypes in a timely manner. Pair programming appeared to work effectively and offer value; however, it is not always practically feasible.

KEY WORDS AND PHRASES: action research, agile methodologies, emergency response, eXtreme programming, pair programming, rapid prototyping, software testing.

INFORMATION SYSTEMS THAT ARE DESIGNED and developed efficiently, accurately, reliably, and meet the intended needs and expectations of the stakeholders are important goals of organizations today. This is especially important for critical systems such as safety and security systems, emergency response systems, and business-critical systems [49]. Meeting system stakeholders' expectations accurately and in a timely manner is a complex task because most systems involve multiple stakeholders with different, often conflicting, needs [14, 22].

There is not one universal methodology for system development that will work for all projects [26]. Practitioners need advice on when to use what methodology [13, 21] and how to operationalize a particular methodology for their development environment. The traditional plan-driven system development methodology requires extensive planning, codified processes, and rigorous reuse [13]. This methodology works best when developers can determine the requirements in advance—including prototyping—and when the requirements are relatively stable [24, 41]. Still, today, the plan-driven model is largely used in practice because of its straightforward and methodical, structured nature. However, in practice, the plan-driven model has a number of key shortcomings that have been widely reported, including the inability to effectively handle changing requirements, and the tendency to significantly be over budget and behind schedule (see, e.g., [11, 13, 22, 41, 45, 46, 50]).

To address some of these plan-driven methodology shortcomings, new system development models were proposed such as the spiral model [12] and agile approaches (e.g., eXtreme Programming [XP], Scrum, and Crystal) [25]. Such new approaches focus on fast deliverables, dynamic management of requirements, and fast iteration and incrementation [41]. Although these new approaches have been proposed and many positive benefits are speculated, there have been few empirical field studies on the results of operationalizing various agile approaches. Therefore, the aim of the research reported in this paper was to assess the viability and effectiveness of one

particular agile method in practice, XP, through an action research study in a mid-sized, R&D, development project for an emergency response system.

In particular, this study investigated the experiences with operationalizing the 12 core principles of XP. The contribution of this paper is fourfold: (1) it illustrates how agile methods can be operationalized to effectively support the development of critical systems, such as emergency response systems; (2) it identifies critical issues that may affect the efficiency and effectiveness of the application of agile methods; (3) in particular, it derives lessons with XP from the field that inform the application of XP in practice; and (4) it identifies key issues for further field research on the application of agile methods.

Literature Review

Overview of Agile Methodologies

IN RECENT YEARS, AGILE SOFTWARE DEVELOPMENT approaches have received a great deal of attention. The Agile Alliance movement was motivated by the observation that software teams in many corporations are entrapped in an ever-increasing amount of processes and documentation. Continued dissatisfaction with the available development methods led to the introduction of various agile approaches (i.e., XP [8, 9], Scrum [42], Crystal [17], etc.). The agile approach focuses on four key ideas [10, 34]:

- *Individuals and interactions over processes and tools.* People are the most important success factor. Often, too much emphasis is placed on coding knowledge and development tools. Instead, team members and their communication with each other should carry a much larger role.
- *Working software over comprehensive documentation.* Software documentation is important, yet information transfer is more effective through the code itself and through human interaction.
- *Customer collaboration over contract negotiation.* Successful software development requires frequent communication and collaboration between the user and the developer, rather than a contract or a statement of work.
- *Responding to change over following a plan.* Long-term project plans are not adaptable; short-term plans provide more flexibility in responding to change. It is considered more effective to devise a detailed plan for a two-week period and a general plan for a three-month time period.

The key differences between the plan-driven and agile approach become apparent when comparing the attributes of seven so-called *home ground* parameters of a software project (Table 1). The home ground parameters often help determine which development approach is best suited for the particular project under consideration [13]. The critical attributes of developers in agile development teams are amicability, talent, skill, and communication [18]. Also, in contrast to the plan-driven approach, the agile approach highly recommends that the developers are experienced in system development and that they are collocated. Further, the agile approach stresses that

Table 1. Agile Versus Plan-Driven Home Ground Attributes

Home ground project parameters	Agile attributes	Plan-driven attributes
Developers	Agile, knowledgeable, collocated, and collaborative.	Plan-oriented, adequate skills; access to external knowledge.
Customers	Dedicated, knowledgeable, collocated, collaborative, representative, and empowered.	Access to knowledgeable, collaborative, representative, and empowered customers.
Requirements	Largely emergent; rapid change.	Knowable early; largely stable.
Architecture	Designed for current requirements.	Designed for current and foreseeable requirements.
Size	Smaller teams and products.	Larger teams and products.
Refactoring	Inexpensive.	Expensive.
Primary objective	Rapid value.	High assurance.

Source: Based on Boehm [14].

customers should be committed, knowledgeable, collaborative, representative, and empowered [13]. They should also be collocated with the project team and dedicated to the project, in contrast to the plan-driven approach.

Plan-driven methods are most effective when requirements are known in advance and stable [42], whereas the agile approach embraces volatile and emerging requirements. Agile methodologies accommodate users altering their requirements once they see and test the system. Agile methods aim to contain the cost of change throughout the development process by explicitly expecting and managing it [25]. Further, in the agile approach, the architecture is produced and refactored as needed, rather than thinking of the ideal system architecture up-front for current and foreseeable requirements [5]. Lower overhead occurs because of reduced documentation and bureaucracy of decisions.

Agile methodologies are intended for smaller, less-complex information system projects with less than ten team members. Large-scale systems that require more than ten team members are better served with plan-driven methodologies. Agile methods can be difficult to scale up to large projects because of a lack of sufficient architecture planning and overfocusing on early results [13]. Moreover, in large, complex systems with less-experienced developers using the plan-driven approach, refactoring is expensive [13]. Agile approaches with skilled developers and small, less-complex systems benefit from refactoring being essentially free.

Finally, the highest priority of agile methodologies is to provide customer value by delivering key features early in the project [10]. Yet agile methodologies are normally not recommended for developing life-critical systems such as life-monitoring information systems or aircraft control systems that require high assurance.

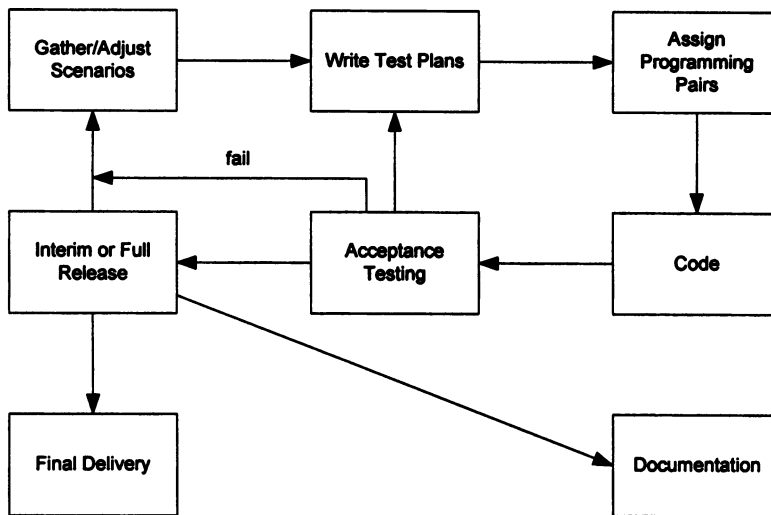


Figure 1. The eXtreme Programming Process

Source: Based on Beck [9].

Overview of eXtreme Programming

One of the more prominent approaches that adheres to the principles of agile software development is XP. XP is a lightweight software development methodology designed for teams of up to ten people that need to develop software quickly in an environment of vague or rapidly changing requirements [8]. Developers constantly review system scenarios that are the highest priority to the business and quickly deliver the functionality described in these scenarios [9]. The XP process can be characterized by short development cycles, incremental planning, evolutionary design, and its ability to respond to changing business needs. It emphasizes productivity, flexibility, teamwork, minimal documentation, and the limited use of technology outside of programming. XP promotes a discipline of software development that is “human oriented” [8, 9].

The XP process (Figure 1) illustrates the frequent iterative development cycles of small releases of functionality with constant user consultation and engagement [8, 9]. Users are responsible for identifying the features that the developers must implement, providing detailed acceptance tests for those stories and assigning priority to them. Developers should implement the stories the users want, in the order the users want, and pass any tests that the users specify. Every cycle begins with gathering user stories representing system requirements. The stories and their associated requirements are simple. Then, the users and developers determine which requirements will be developed in the next iteration by prioritizing the stories. Next, test plans are written prior to coding, followed by assigning programming pairs to specific tasks. Upon completion of the code, each section of code is tested according to the defined functional test cases. The users must also be engaged in testing. If the acceptance test fails, the users and developers will meet again to adjust the user stories and the process will

repeat. Once the testing is completed with expected results and the results are accepted by the users, the enhancements to the system are released. New enhancements are released in two-week cycles. Following the release, documentation is completed and a final delivery of the system is made to the end users.

The values of XP are implemented and enforced by the use of 12 principles, which are listed in Table 2. These principles are core to the use of XP. Research suggests that following these principles leads to several advantages over traditional software engineering methods [8]. On the other hand, critics of XP state that the 12 principles are highly interdependent. Each principle cannot stand on its own because of its reliance on at least one other principle to support it. So, tailoring the XP principles can be problematic [47].

System metaphors are a powerful way to relate a difficult idea in an unfamiliar area by defining a concept or feature using a simplistic representation [5]. This gives the developers and users a common ground of understanding of how the system is expected to perform.

XP recognizes that not everything is known at the beginning of the project. The XP planning game makes a rough plan quickly and refines the plan as things become clearer. Planning is continuous throughout the system development process [6].

Small releases and the continuous design process allows for frequent review of the system by the developers and the users [43]. XP depends on rapid feedback from the customer to establish the accuracy of the functionality of the scenario that is being implemented [9]. This constant and quick feedback helps identify errors, supports ongoing usability tests [33, 38], and builds trust between the users and the developers.

The simple design approach enables frequent changes to be made to the system easily and encourages design experimentation. On the other hand, XP is sometimes seen as a chaotic method that lacks discipline. Some project managers view XP as dangerous and unpredictable because, to them, it appears to neglect planning and controlling that is done in large-scale or previous projects [32]. Moreover, the fact that XP does not maintain specific requirements from the start of development makes it difficult to guarantee that the customer's needs are actually satisfied [36]. XP lacks up-front design and investment in life cycle architecture, and relies primarily on tacit knowledge of user and informal communications [15].

According to Beck [9], testing should occur every time a code change is made, and automation of testing is recommended for efficiency. The implementation of an automated testing method in which unit tests are developed before actual code is written has shown to increase confidence in functional and system reliability [35, 44].

Refactoring is a technique used to improve code without altering functionality, and its goal is to produce programming units with a strong internal structure. These programming units are generally more reusable, object-oriented, pattern-oriented, maintainable, and simpler [44]. Further, refactoring reduces the complexity of the code by removing unused code and implementing patterns [20]. Refactoring helps developers respond quickly to changing requirements [44]. However, there are mixed opinions on the cost of refactoring [13]. Refactoring is expected to be done on an ongoing basis. This may lead to extra cost and to higher system overhead.

Table 2. eXtreme Programming 12 Key Principles

Principle	Principle overview
1. System metaphor	Simple shared story of how the system works to give both developers and customers common ground.
2. Planning game	Planning is continuous and progressive. Developers estimate the cost of the candidate features, and customers prioritize features based upon cost and business value.
3. Small releases	Frequently releasing simple systems, and releasing new versions on a very short cycle (one to three weeks).
4. Simple design	Keeping the system design as simple as possible, and finding and removing extra complexity.
5. Testing	Writing and running unit tests as code is written to make sure the system is working properly. Frequent user acceptance tests to ensure the system is fulfilling user requirements.
6. Refactoring	Used to improve code without altering functionality. Focuses on simple, clean, nonrepeating code that can be easily changed.
7. Pair programming	Two programmers developing production code at the same time on one machine.
8. Collective ownership	All developers responsible for all code, therefore changes to the code can be made by anyone at anytime when necessary.
9. Continuous integration	Integrating new changes with current code as they are completed to detect system failures as soon as possible.
10. Forty-hour week	Developers keep a normal work schedule to remain productive and interested in the project.
11. On-site customer	A customer sits with the development team full time.
12. Coding standards	Developers write all code in accordance with the standards agreed upon by the team to ensure that communication is made through code.

Source: Based on Beck [9].

Pair programming is where two people develop tests and create code side-by-side. The idea is that there is a free flow of ideas, a richer experience base, and early defect detection resulting in improved overall quality of the tests and code [36]. There is an ongoing debate on the value of pair programming [30]. In some cases, empirical research has found that pair programming produces higher-quality code at a lower cost [35]. Further, it is argued that the increase in the cost of development using XP is offset by productivity gains (e.g., a pair of programmers has a higher development speed than a single programmer) and increased quality of the code through the continuous checking of the code against test cases by the second programmer [19, 20]. Yet pair programming could also increase system overhead if higher code production and quality code does not occur. In addition, in one case study, it was a hard concept

to implement: engineers were not comfortable working as pairs [39]. Despite the resistance, quantitative improvements in productivity occurred [39].

An early XP success story described Chrysler's compensation system [2]. Another study found that three principles—planning game, collective ownership, and collocated customer—worked best and contributed most to the success of an online telemarketing system [28]. XP maintains collective ownership where everyone owns all the code and coding changes are made by anyone at anytime. Close team interaction, shared goals, and limited time invested in elaborate designs provide for an environment where there is less defensiveness and territoriality [4]. Using XP as the development methodology has shown to increase team morale [39].

XP is said to improve the overall product stability and maintainability [39]. It is believed to enable effective software development by allowing organizations to deliver and change requirements quickly during the software engineering process. Advantages of XP over conventional practices include lower management overhead, higher team productivity, happier customers, and shorter release cycles [15]. Critics express various objections, including little detailed requirements, lack of design clarity, the need to do constant refactoring, the duplicated human resource requirements of pair programming, and requiring a customer to collocate [47]. A summary of the benefits and shortcomings of XP is shown in Table 3.

Method

THE AIM OF THIS RESEARCH WAS TO EVALUATE the XP method in practice, to make adjustments where needed for each iteration, and to reflect on those adjustments. Therefore, action research was considered the most suitable research method. Action research, a qualitative approach, allows researchers to gain a rich understanding of people and the context in which they live and work [37]. Below, we elaborate on our method and highlight the collection and analysis of our data. We also discuss the role of the researchers in this study.

Action Research

Action research has the dual intention of improving the practice and contributing to theory and knowledge [3, 7, 16]. It is often credited with producing relevant research results [7]. We followed the model proposed by Zuber-Skerritt [51] (see Figure 2), which states that an action research study may consist of four activities that can be carried out over several iterations. "Plan" concerns exploration of the research site and the preparation of the intervention. "Act" refers to the actual intervention made by the researcher. "Observe" concerns the collection of data during and after the actual intervention to enable evaluation. Finally, the "reflect" activity analyzes the collected data and infers conclusions that may feed into the "plan" activity of a new iteration.

Action research was selected as our research approach for several reasons. First, action research is especially appropriate to address exploratory "how-to" research

Table 3. Summary Advantages and Disadvantages of eXtreme Programming

Benefits	Shortcomings
Quick prototype production	Scalability issues
Iterative process	Overfocusing on early results
Frequent feedback from user	Pair programming no cost-effective
Encourage design experimentation	"Test-drive" approach adds development time
Increased user satisfaction	Lack of specific requirements to ensure user satisfaction—short-term requirements
Refactoring	Unstructured
High code production	Unpredictable
Quality code	Neglected planning
Team ownership of code	Required experienced developers
Cohesive developer environment	Access to colocated users
Flexibility	Lack of documentation
Lower overhead	Higher overhead
Most beneficial for teams with less than ten members	Not recommended for team with more than ten members

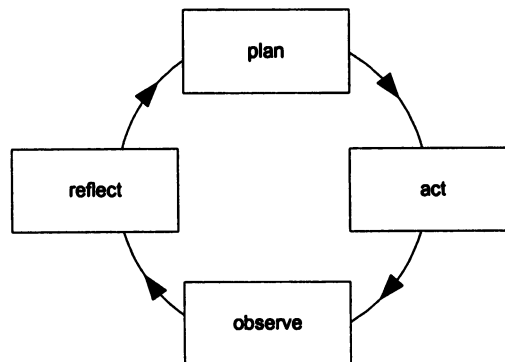


Figure 2. Action Research Model

Source: Based on Zuber-Skerritt [51].

questions [7, 29]. Our research aimed to explore and develop ways in which development projects could benefit from using the XP approach. Action research is particularly appropriate to produce knowledge that guides practice [1, 7]. Second, the continuous design and evaluation of a software engineering process was considered too complex to be studied in a constructed setting. In fact, it has been argued that researching new systems development methods represent the ideal application of action research [7]. Third, action research is well suited for continuous learning. It allows researchers to continuously evaluate and improve their problem-solving

techniques or theories during a series of interventions. In our research, we produced several iterations of the system prototype. The experiences from each prototype resulted in changes to the XP processes that were effectuated immediately. Finally, as far as we knew at the initiation of the project, little research had been conducted on the use of XP in the telehealth application development environment. Therefore, it was considered most appropriate to study this issue in a real-life problem situation.

Data Sources and Analysis

As is common in many action research studies (see, e.g., [7, 23, 29]), we used multiple sources of data, both quantitatively and qualitatively, to enable a rich understanding of the situation at hand:

- *Direct observation.* The researchers took notes of incidents, participants' remarks, and events that conveyed critical information.
- *Group interviews.* The researchers held biweekly status meetings with the project team that provided an open forum to discuss what was going well on the project and where adjustments were needed.
- *Usability evaluation.* After the release of three key prototypes at different points in time, full-system usability evaluation was done.
- *System documentation.* Test plans and release notes were stored electronically for each prototype.
- *Timeline.* A timeline of the prototype iterations was documented.

The study took place from October 2002 until August 2004. During this period, user scenarios, release notes, prototype requirements, and meeting minutes were kept in an online workspace for all team members to share. All team members, developers and users, contributed to the online workspace. The variety of data sources enabled comparison and contrast of the collected data. Given the exploratory nature of the study, we analyzed the development process of the system at each iteration, monitored the feedback from the users, and reviewed the system documentation and outcomes in which we identified areas of improvement and refinement. In particular, we investigated user satisfaction through usability testing, project team satisfaction through informal one-on-one meetings with the project manager, and made note of critical insights on the process and quality of the XP method.

Role of the Researcher

The role of the researchers was that of observer and participant at the same time. Researcher participation was twofold. First, one of the researchers also served as the project manager. This included assigning development tasks, coordinating meetings, budget oversight, and technical guidance. The other researcher assisted in the usability testing. Both researchers facilitated data gathering activities that involved the team members. Both reflected on the data and analyzed it.

Emergency Response System Project

OUR STUDY FOCUSED ON THE emergency response system (ERS) project. This project represents a successful effort to improve U.S. biosecurity preparedness for public health concerns through information technology (IT) support. Initially, the ERS application was a conceptual idea of one of the key stakeholders (users). It was expected to be a new and innovative laboratory response system to both man-made (terrorism) and human disease emergencies where human life is at risk. The developed ERS supports lab personnel to communicate and collaborate regarding health emergencies transcending individual labs—for instance, concerning the diagnosis of suspect specimens, recommending protocols, or ruling out possible diseases such as smallpox. The ERS allows for immediate real-time response to alerts and secure communication of critical information to those who need to know. The initial anticipated user base of the ERS project consisted of public health laboratory experts in a Midwestern state. Currently, other states have shown interest to have the developed system installed in their key public health labs as well.

Selecting the eXtreme Programming Approach

There were several reasons why XP was chosen as the system development methodology. First, the requirements of the system were initially not well established, evolutionary, and changing. No system existed to evaluate or reengineer. Second, the team members had somewhat varying levels of technical skills and system development experience. For that reason, working as “pairs” was attractive. The IT project team and the customers were not well acquainted; in fact, they had never worked together, or even knew each other, until this project. Consequently, it was important to establish trust and assess the level of commitment between the two groups at the onset of the project. The customers were a little leery as to whether the IT team would deliver what they promised in a reasonable amount of time. So, the ability to produce a working prototype with a graphical user interface within a few months was essential. Finally, XP was selected because of the promise of minimizing overhead costs. As shown in Table 4, the project environment was congruent with most of the “home ground” rules for XP.

The Team Profile and Development Project Environment

The development project was housed at the authors' university and funded through a competitively awarded state grant for innovative, applied research in IT. The project team consisted of one part-time project manager (5–10 hours a week), one senior technology research expert to help solve technical problems (5–10 hours a week), three part-time users (totaling 4–20 hours a week depending on the week), and two to six part-time developers (averaging a total of 80 hours a week). Developers were not asked to work overtime. The technical expertise varied among the IT project team members both in expertise depth and span of knowledge. The developers were graduate

Table 4. Comparison of Agile Home Ground and ERS Parameters

System parameters	Agile attributes	ERS project attributes
Developers	Agile, knowledgeable, collocated, and collaborative.	Agile, varying technical skills, collocated, and collaborative.
Customers	Dedicated, knowledgeable, collocated, collaborative, representative, and empowered.	Dedicated, knowledgeable, collaborative, representative, and empowered (not collocated).
Requirements	Largely emergent; rapid change.	Evolutionary.
Architecture	Designed for current requirements.	Designed for current requirements and lightweight architecture.
Refactoring	Inexpensive.	Inexpensive.
Size	Smaller teams and products.	Project team size: two to six developers. Small product—limited integration requirement with enterprise systems.
Primary objective	Rapid value.	Customer satisfaction.

students with industry IT experience and undergraduate seniors with strong applied IT skills. The project manager and senior technical research expert had extensive industry experience with systems development. The team had never used the XP process prior to this project. The users of this proposed system, public health experts, would be interacting with the ERS on a routine and emergency basis. Their combined years of experience in clinical laboratories were over 70 years so they had the experience to make knowledgeable and informed decisions [40]. They also had a vested interest to see the project result in a successful implementation of the ERS in the field.

The IT development team shared a 15' × 23' room that had desk space for each developer to have his or her own work space and computer with T1 Internet access. Also, the test servers, the test clients, and all the hardware needed to build the prototypes were located in this room. A large whiteboard was available to draw designs, and so on. A community refrigerator was shared by the team. Guest chairs and a moveable table were available for team members to work together.

Emergency Response System Application

The goal of the ERS was to address critical public health communication and biosecurity needs for the State Public Health laboratories. The ERS application needed to be a secure, dedicated, Web-based network system that connected various clients

who were located in remote and rural locations. The ERS had the potential of being placed in 85 laboratories across the state, so it was necessary for the ERS application to work in a distributed environment. A client/server architecture was selected for the system using the Linux operating system and open source when applicable. Customized application code was also developed internally. The ERS server consisted of 247 files and 23,130 lines of code that the ERS project team maintained or had developed. The client consisted of 160 files and 2,003 lines of code. Other open source files were used on both the client and server and, as of this writing, the ERS team has not had to alter and maintain them.

The ERS was a new, innovative system that was expected to push the envelope on leveraging the latest technology, versions of open source, and newest hardware available. Both the hardware and software were engineered for this project. In other words, the project team researched the hardware that met the user specifications and assembled the system as well as software construction. The physical ERS consists of a computer terminal; a flat screen/speaker unit; a mini, virtually indestructible keyboard; a high-resolution digital camera; a biosafe Plexiglas specimen container; and a microscope interface kit attached to a microscope. The high-resolution digital camera captures macroscopic images of culture plates and the microscope interface kit works with the microscope to capture microscopic images.

The ERS has several sophisticated and complex functions. Some of the significant functions are processing alert transactions, managing alert transactions and images in a repository database system, providing real-time electronic consultation between the various users, monitoring the system network status, and providing an easy to learn and useful graphical user interface. In addition, a critical nonfunctional requirement was implementing HIPAA (Health Insurance Portability Accountability Act of 1996)—compliant security measures (i.e., encryption, secure socket layer [SSL], secure shell [SSH]).

The ERS had the unique situation that the technical requirements of the system were as important, if not more important, than the functional requirements. For example, because this system was intended for emergency alerts and notifications, response time was important. Also, the quality of the image that was captured by the digital camera and transmitted needed to be a high resolution so that real-time consultation could take place. An important consideration of the development of the ERS was that it could potentially save lives. Therefore, the system needed to have high reliability and be extremely accurate.

Developers were truly working in an R&D environment (i.e., researching and evaluating various open source and hardware technologies), which sometimes created an unstable technical environment. In several cases, the developers were researching hardware and software that had been on the market for a few months. Examples of this include particular mini-motherboards, Firefox, Mozilla, and Jabber.

The system development team had to deal with changing software and hardware technical requirements. For example, as mentioned earlier, the camera functionality was critical. In a later iteration, the users requested that they be able to pan the client cameras and control zoom functions remotely. This request was stated as a scenario,

and then the project team researched cameras that had the functional hardware capabilities and developed the user interface.

Sometimes, the team came upon roadblocks, forcing the proposed solution to be scrapped, and the problem had to be tackled in a different way. For example, when the team chose to use open source for the electronic real-time consultation, there were several options available. The original open source selected worked for the first several scenarios, but, as the system evolved, it became unstable, so another open source option was researched and implemented in its place.

The “footprint” of the ERS needed to be minimal, as the environment where the client ERS were being placed was cramped for space. The hardware and software costs were required to be kept to a minimum as well as the system development overhead.

System security was a critical development requirement that the ERS development team had to consider in every implementation of the scenarios. Another systemwide requirement the ERS project team had to contend with was developing a scalable system that could support rural and remote laboratories. The telecommunications infrastructure and IT expertise varied greatly among the potential client locations. Occasionally, the requirements of the telecommunications infrastructure were unknown until the time of deployment.

Results

IN THIS SECTION, WE FIRST DISCUSS how the 12 XP principles were operationalized in the ERS project (see Table 2 for an overview of these principles). We then summarize to how the ERS project followed the XP methodology and what specific modifications were made to make it more effective.

Operationalizing eXtreme Programming Principles

System Metaphor

At the onset of the project, there was an organizational meeting for the ERS project team. All the key stakeholders and IT project team members met and introductions were made. The users provided an overview of how they envisioned the usage of the ERS and their expectations of the communication between the labs that the ERS would provide. One of the first tasks in the project was to create high-level scenarios for different use cases to be used for the development of the early prototypes. High-level scenarios are written at a coarse level of detail, outlining the basic activities as envisioned the by the user. These scenarios then were prioritized by the customer. These user/developer–shared scenarios (system metaphors) gave the developers and the users a common understanding of how the system should function. Scenarios were continuously shared by the users as the system development continued. In total, over 50 scenarios were developed. Initially, there were only high-level scenarios (e.g., “emergency alert pager” and “microscopic image capturing”), which were subse-

quently broken down into detailed scenarios. The scenarios were not only useful for the initial development of the ERS but also informed test plans, usability testing, user training, and field demonstrations to key external stakeholders, including the state governor and state chief medical officer. The value of creating these scenarios was clearly demonstrated during a visit by the state epidemiologist. A scenario on the handling of emergency alerts had been developed that specified the protocol and maximum response times. During his visit to an operational ERS prototype site, the epidemiologist checked the system and sent an emergency alert. As the protocol stated, the lab that he was visiting was contacted within the required 15 minutes by the state public health laboratorian on-call to support the ERS.

Planning Game

After the initial discussions with the users, the development team began to research what type of hardware technology could be used for the ERS development effort. This helped clarify some of the early requirements. Still, at times, the requirements were vague and incomplete because the users did not know what technology existed and how it would function. Producing multiple iterations of the prototype helped to continually refine the functionality of the system. For example, the functionality of the camera went from capturing an image to remotely controlling the camera. Another example concerned the determination by the development team of the type of camera needed to produce the minimum required image quality to enable consultation and diagnostics.

Planning of new features occurred at multiple venues. Each quarter, all key stakeholders met to brainstorm and identify new features and prioritize their importance. Additional features were also identified outside of the quarterly meetings. At weekly status meetings, tasks statuses were updated verbally and new tasks assigned based on the priority high-level scenarios defined by the key stakeholders. Users attended the weekly status meetings and provided feedback. On a daily basis, project team members discussed the tasks they wanted to accomplish.

During the first year of development, three usability evaluations of the system were also completed by different sets of stakeholders. The recommendations from the usability evaluations were also included in the tasks that were prioritized. For example, the “sending alert” window initially had very basic and few attributes. After the system was placed in the field and users’ feedback was collected, several additional attributes (text boxes and drop-down text boxes) were added that described the image that was being sent.

Small Releases

A working prototype was functional within a few weeks after the initial meeting. This first prototype only included about 5 percent of the intended functionality, as it merely served as a proof of concept. The team continued to produce many iterations of prototypes throughout the project. As shown in the timeline displayed in Figure 3, 10

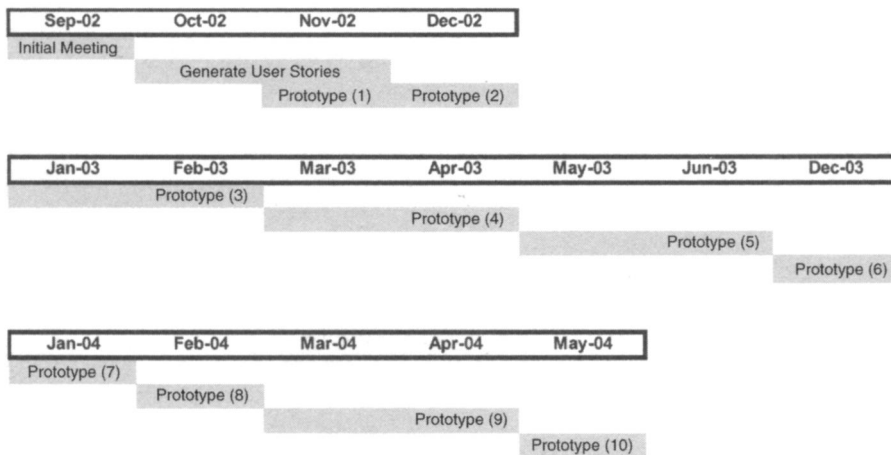


Figure 3. Prototype Release Time Line

major milestones of prototypes were produced in 21 months. Smaller release cycles were completed internally for the development team to keep integrating each new functionality. In total, there were over 30 smaller iterative release cycles.

Developers released new features on a weekly to monthly basis, depending on the users' schedule and the developers' ability to support the system if something went wrong. Developers set the dates for minor deadlines with the guidance of the project manager. Major deadlines were set by the project manager with input from the developers. Deadlines were reasonable. In most cases, the team followed the strategy of small releases. In a few cases, the new feature was complex and affected a large percentage of the system and would be considered a large release. This occurred more often in the second year as the system became more sophisticated.

Rapid prototyping of small releases appeared to be a key factor in the early "buy-in" of the users. Asking the users to share stories and then producing a prototype in a matter of weeks that functionally performed the tasks was a major advantage. The early user feedback on small prototypes reduced the amount of rework and the possibility of the development team going down the wrong path. Also, by having a prototype up and running quickly created strong momentum and group synergy. In particular, it appeared to stimulate the users to share more ideas. The users initially seemed somewhat afraid to ask for new features for fear it was too difficult or time-intensive to implement. Once they realized that it was all right to request changes or additional features, they opened up. Further, the iterative prototyping also allowed the user to have a better understanding of the technology available (e.g., the quality of the image taken by the camera, the durability of the keyboard, the size of the flat panel, the footprint size of the unit, etc.). It also provided a platform to define unfamiliar terminology, both technical and laboratory terminology.

Simple Design

Developers were encouraged to keep the design simple. One of the main standards of the system was to construct a very thin client, which, by definition, required simplicity. Another way this was ensured was through usability evaluation of the graphical user interface with the goal of creating a system that was easy to use and simple to learn. In this sense, the XP approach proved to be adaptive to the development, integration, and implementation of new technologies. By keeping the design simple, changes that required new or different technology (i.e., camera, browser, open source of instant messaging) were transitioned quickly into the system. Several new technologies were coded, tested, and deployed. In some cases, new technologies replaced old technologies. For example, the ERS is now on its third camera prototype. In other cases, initial parts of the system were simplified to improve performance and maintainability. For example, the team developing the system status module wrote a tightly coupled method containing hard-coded ERS location names inserted in over a dozen other modules. This led to a maintenance nightmare. Every time a new location or feature was added, the location names had to be updated. A second team of programmers refactored the system status module, using global variables that accessed an index where updates would only need to be made in one place. Then, maintenance issues relating to the module dropped to virtually nothing.

Testing

Another principle of XP is writing the test plans before coding [27]. In the first several iterations of the prototypes, the developers, with the help of user feedback, wrote the test plans prior to the coding. As the development effort increased, test plans were written in parallel of the coding. For this purpose, the creation of new test plans and regression testing was assigned to a nondeveloper project team member. This allowed developers to concentrate on the development effort as test plans were becoming more complex and more tests were required for each new iteration. It also added a fresh perspective on testing and resulted in better utilization of the human resources on the team; it appeared that one team member did not have the expected strong technical skills, yet had a good grasp of the system and good communication skills. As regression testing was becoming more and more time-consuming and the system was now in production, her assignment was redirected to focus on testing. This made some of the developers happier as they did not have to do regression testing. Finally, having a designated tester allowed the project to incorporate the users' feedback on prototypes more swiftly. Rather than having to update test plans first, the developers could get to the new development tasks immediately.

Refactoring

Refactoring is an important XP principle [8]. Developers reviewed the code for efficiencies when they added more functionality. In one case, refactoring was extremely important in shortening the time it took to load certain HTML pages. Feedback from

the users indicated that system navigation was too slow, especially in the image capturing process. When a developer learned about cascade style sheets, he realized that this could help eliminate a lot of redundant code and would speed up the navigation process. The resulting performance improvements were very noticeable. Other examples of refactoring were when usability heuristics standards were established, global variables needed, and new coding standards identified.

Pair Programming

The amount of pair programming varied during the development of the system. In the first year, the IT project team was made up of two to four developers, and they worked in pairs on the coding. As the system became more sophisticated (year 2), two additional 20-hour-a-week developers were added to the project team, and one of the developers converted to a full-time team member. For the most part, "pair programming" continued. However, there were times when developers' work schedules prevented pairing up, so one person continued working on the task solo or was assigned individually. Also, as the system moved to production, there were times that doubling up on tasks did not work because of maintenance and deployment tasks that needed to be accomplished simultaneously. This was due to the size of the project team. Moving away from pair programming did not appear to harm quality as the testing process stayed the same. Also, developers still discussed the changes they were making and got a second opinion when needed.

Developers were "paired" differently for different assignments. The pairing depended on the developers' technical skills, domain knowledge, and availability. In some cases, pair programming occurred without being assigned. It was done on the developers' initiative. This happened when either (1) a developer needed additional technical expertise or (2) due to time constraints, the task could be completed faster by two individuals. When tasks were assigned that the development team did not have previous technical knowledge, it was often requested by the developers that two people work together on coding the solution.

Pair programming indeed proved to be most beneficial when developers had varying levels of technical expertise. By cross-training and redistributing the technical expertise on the team, the overall technical expertise and system domain knowledge increased quickly. For example, only one developer was familiar with PHP (Hypertext Processor), so another developer was assigned to work with him. This brought the inexperienced developer up to speed quickly. An example shows that, in any case, one of developers in the pair has to be experienced: we assigned two developers to add encryption to the messages being sent (this was something they had not done before). Several days went by and little progress was happening. It seemed that neither developer wanted to confess that they were having problems getting things to work correctly. They kept saying they almost had it done. We realized then that when progress was not being made as rapidly as in the past, it was because they did not know how to do it and were stumped on some problem.

In a few cases, pair programming caused problems with accountability on task

completion. There was not “one” person accountable to get the task completed or to report back when they were “stumped” technically and were not making progress. Stephens and Rosenberg [47] report this same problem. There was a tendency to hide or not disclose that the pair team needed additional technical expertise. At times, neither member wanted to admit his or her lack of technical expertise, so, in some cases, peer pressure caused problems. This was addressed by assigning one of the pair programmers to be the lead and the reporter of the progress of the task. Lead programmers alternated throughout the project.

It was observed and expressed by those involved that the project team became a very cohesive group. This proved valuable when certain milestones needed to be met; everyone helped each other and was dedicated to completing the task on time. It appears that this could partially be attributed to the close working relationships of the developers required with pair programming.

Collective Ownership

The team worked with the philosophy of collective ownership of the code. Different pairs of developers worked on the same code at different times. All developers were responsible for the code. One benefit of collective ownership became obvious when the team walked through the entire project code during a security audit. There was not a sense that one team member’s code was being examined. The entire team accepted the responsibility when concerns were encountered. This resulted in a very productive meeting.

Another benefit of collective ownership is the shared knowledge of the system developers since everyone works on multiple areas of the system at different times. Throughout the project, all developers had a strong desire to understand every part of the system. New ideas and features surfaced that increased creativity. Developers did not seem to be worried about stepping on each other’s toes, because one team member “owns” a certain aspect of the system. Also, better project resource allocation occurred. Tasks could be worked on according to priority and not availability of the team member who “owned” that aspect of the system.

A potential pitfall of collective ownership is that if everyone owns the system, nobody owns the system. In other words, nobody feels responsible for final results. This risk was mitigated in the ERS project by always making sure that a lead was assigned for each development task. Also, hardware/software installations were promised to users, and not meeting these deadlines would embarrass the team. In other words, “saving face” was important, especially since the developers and users knew each other personally—users knew who was making the changes and if they were doing a good job.

Continuous Integration

New code or updates to the code were integrated into the production base code continuously. Ideally, integration should occur after each task is completed. Usually, the developers integrated the new functions with the base code and then performed regression testing. However, occasionally, new features that took longer development

time were isolated and then integrated with the base code after the new feature worked correctly independently.

Forty-Hour Week

The IT developers in the project had various levels of expertise and worked 20 hours each week. The users in the project were available as much as possible, but they did have other tasks that took priority. These two factors probably affected the speed of the prototype development and implementation. The implementation dates were partially set based on the users' availability. Had the developers worked 40 hours each week and the users could interrupt their other work, the time line would have been considerably condensed, perhaps by as much as 40 percent.

On-Site Customer

One of the home ground areas of agile methods is the collocation of the users [13]. Although the users were committed to the project, eager to answer questions, and share their ideas on the features of the ERS, they were extremely busy microbiology laboratory professionals. Their other responsibilities and laboratory equipment did not permit them to be collocated. Because the users could not be on-site, e-mail was extensively used and served as mini-documentation. Also, weekly face-to-face team meetings with all users occurred throughout the project. Weekly meeting minutes and agendas were assigned on a rotating schedule throughout the project team and e-mailed to all stakeholders.

It did not appear that not having a collocated user negatively affected the system development progress. Even though the users had many distractions, the meetings to "gather and adjust user stories" were effective and efficient. Producing a working prototype early in the development process had a considerable positive effect on user ownership. User ownership and dedication to the system occurred very early in the development process, and, in fact, as time went on, the ERS project became one of the users' priority distractions. The users were extremely pleased with the functionality of the system and even more impressed with having a functional system to evaluate at such an early point in the development process.

Coding Standards

Team-agreed-upon coding standards were followed. When the developers worked in pairs, standards were discussed and then implemented. This helped keep things consistent and provided additional guidance during refactoring efforts.

Similarities and Modifications

Overall, the ERS project followed nine of the 12 XP principles and modified three principles. A summary is presented in Table 5. In the ERS development project, the

Table 5. Summary of Adoption

Practice	Adoption
Planning game	Fully
Small releases	Fully
Simple Design	Fully
Testing	Modified
Refactoring	Fully
Pair programming	Modified
Collective ownership	Fully
Continuous integration	Fully
Forty-hour week	Fully
On-site customer	Modified
System metaphor	Fully
Coding standards	Fully

principles that worked the best were small releases, simple design, and collective ownership. The principles that were found hardest to implement were coding standards, refactoring, and writing the test plan first. This contrasts with another study that found that the principles that worked best were the planning game, collective ownership, and customer on-site. The principles that were found the hardest to introduce and were not so successful were small releases and testing [28].

We made several modifications and extensions to the XP principles. First, *testing* initially started with the methodology of writing the test plan first and then the code. This continued for a while until the system became more complex and extensive regression testing was required. Second, *pair programming* was modified by assigning a lead developer to report on the status of the task, and pair programming was not used sometimes when the system was in production and resources needed to be redistributed.

Third, we introduced *systemwide code walk-throughs*. A shortfall was not having an all-project team code walk-through so that project coding standards could be enforced. If pair programming is done with experienced programmers, then code walk-throughs may not be necessary because they would be more familiar with coding standards, but inexperienced developers require more oversight on following unfamiliar standards. The developers had strong technical skills, but did not have a lot of experience developing enterprisewide systems. The project experienced turnover of one main developer at the end of the first year. At that time, the project manager required a code walk-through of the entire system to share knowledge with newly hired developers who were less familiar with the system and to address coding standards. The collective ownership of the code proved valuable in the code walk-through for two reasons: no egos were bruised when questions were asked about the reason for a coding design decision, and, if a better way of doing the process was identified, the developers were receptive.

Fourth, we appointed a *technical writer*. In the implementation phase, the ERS project needed more system documentation than what was produced during development.

The project manager addressed this issue by hiring an individual that was assigned the sole task of writing the technical documentation. This individual was not one of the developers. The technical writer interviewed the developers and then wrote the documentation. This kept the developers focused and did not interrupt their code production.

Fifth, as described above, the on-site customer principle was not adopted. However, one can argue that having an on-site customer is but one way to achieve a particular outcome—early and continuous user involvement. This outcome was, in fact, achieved through frequent and focused meetings and continuous communication through e-mail and phone. Therefore, adoption of the principle is listed as “modified” in Table 5.

Finally, *usability testing* was added as one of the steps in the XP process. Usability testing was done when there was a fully vertical and horizontal functioning prototype and focused on system consistency, terminology, interface design, learnability, memorability, visibility of the errors, and aesthetics. Further, usability testing identified additional functional and nonfunctional requirements that improved the ease of use and ease of learning the new system. This was another way in which new, innovative features were identified. Various stakeholders and usability experts participated in the usability testing workshops.

Discussion

IN THIS SECTION, WE DISCUSS THE FINDINGS of our study. We first address the four key ideas behind agile methodologies (see the section Overview of Agile Methodologies) and how they applied to the ERS project. Then, we elaborate on some key lessons learned.

Key Agile Ideas

The four key ideas of the agile approach manifested themselves at various moments during the project.

Idea 1: Individuals and Interactions Over Processes and Tools

Methodologies, management techniques, and technical approaches are all important to the success of the system development, but without the right people and team member interactions, the success of the system is in jeopardy [48]. This is one of the key points of the agile methodology: value individuals and interactions over processes and tools.

Value of the people. In the ERS project, the team members’ communication and interaction with each other played a large role in the successful completion of the project. Team members were creative and committed to completing tasks as promised and to delivering quality code. The users were collaborative, representative, authorized, com-

mitted, and knowledgeable, which have shown to be the attributes that make projects successful [48].

The availability of whiteboards, open spaces for discussion, and shared and close proximity computers facilitated the impromptu interaction and discussions. Morale among team members was high. This was evident, for example, when the team would pull together to solve time-critical problems or prepare for the installation of new clients. They performed these tasks with enthusiasm and on their own initiative. Nonwork interaction and socializing also occurred. Team members organized group lunches, played recreation sports, attended social activities, and so on.

Value of the tools. Development tools were used on an as-needed and when-needed basis. For example, the team implemented CVS (Concurrent Versions System) for version control and used integrated development environments (IDEs) for development. However, project management documentation and tools took a less formal approach. Spreadsheets and weekly status reports were used rather than comprehensive project management tools.

Idea 2: Working Software Over Comprehensive Documentation

The importance of system documentation for software engineering is being debated in the literature. In fact, research results suggest that the software industry and academia may be overemphasizing the importance of document maintenance relative to a developer's tolerance of outdated content. In one study, conflicting results were found on the effectiveness of not-so-up-to-date documents. Developers stated that up-to-date documents are very important, yet they also concurred that not-so-up-to-date documents could still be effective [31]. Agile developers are more likely to agree that documentation should be brief and to the point [31].

Document content can be relevant even if it is not up-to-date and that documentation is an important tool for communication and should always serve a purpose [31]. Documentation, like communication, can then be evaluated based on its ability to impart knowledge in its audience; something that need not be up-to-date and consistent (just good enough to serve its purpose) [17].

Although the ERS project manager was initially uncomfortable with the small amount of documentation because of her past experience with plan-driven methodologies, she soon realized that this was not an issue. Because of the reduced amount of documentation, several benefits to the project resulted. The progress of the project did not get hung up on writing and reviewing documents and requiring user sign-offs at every stage of development process or for every major or minor decision. This streamlined the development process. Decisions were made verbally and the developers acted on them immediately. Possible reasons why the sign-offs were not an issue was because of:

- the early established level of commitment between the user and development team,

- the responsiveness of the development team by producing new prototypes with the functions the customers requested in a short period of time,
- the need for the users' time to be used as efficiently as possible and for the user to not be inundated with documents to review, and
- e-mail served as a documented reference point for some decisions.

Initially, system documentation consisted of scenarios, test cases, and the self-documented code. As the prototype became more fully functional and was deployed to field testing, an accompanying training manual was created. In the second year, a technical writer was hired to compile a one-stop comprehensive system documentation notebook that included the hardware specifications, backup and recovery procedures, system overview charts, data flow diagrams, test plans, client implementation plans, security policy, and so on. The developers were asked if they wanted to write the system documentation; they declined and agreed a technical writer would be fine. Having a technical writer focus on the necessary system documentation needed for future reference proved to be a positive decision.

Idea 3: Customer Collaboration Over Contract Negotiation

User/developer collaboration provided valuable feedback on the system on a regular and frequent basis. Rather than relying on written requirements, developers and customers jointly discussed the scenarios and the next round of functions that would be added to the system weekly. Quarterly status meetings were held with the director of the user department, monthly status meetings were held with the assistant director, and weekly meetings were held with the user technicians. The user laboratory technicians were also available by phone and e-mail and for impromptu meetings as needed.

Idea 4: Responding to Change Over Following a Plan

"When we build plans, we need to make sure that our plans are flexible and ready to adapt to changes in the business and technology" [34, p. 11]. The course of a software project cannot be planned very far into the future because the business environment changes, customers are likely to adjust the system once they have seen an example, and estimating how long it will take to do requirements is only an estimate; the best-guess estimate can be used for a plan, but estimates change [10, 34].

The project manager had a general plan for each quarter and then went back and worked with the team each week to have a two-week plan and sometimes even a weekly or daily plan. A large, yearlong calendar divided by days that could be written on with markers was posted on the wall in the development team's work area. It was used for planning the next iteration and deployment of new clients. This multilevel planning strategy worked well when responding to changes. The two-week plans and deadlines were adjusted as needed when technical or user requested changes occurred or technical roadblocks surfaced. Yet large milestone (e.g., alpha prototype, field prototype) deadlines were achieved. In this sense, the XP approach did not suffer

from the scheduling deceptions that so often thwart projects that adopt the planned approach.

Lessons Learned

Several lessons were learned from this study. There is value in implementing individual small releases singularly rather than batching them: if a problem does occur after the implementation, it is easier to pinpoint the cause of the problem. For example, some problems (i.e., new browser, security features, and network changes) cannot be completely tested prior to deployment in a distributed environment where updates to the system are done remotely. When implementing a release with several code changes, new functions, and enhancements, it is problematic because time has to be spent on eliminating which code changes did not cause the problem. Single feature implementation pinpoints the root of the problems easier.

In the ERS project, we spent as much time testing as coding. The visibility and criticality of the system demanded regression testing often. Having a dedicated regression tester as part of the XP project team, when test automation is not possible (i.e., a distributed system), allowed the developers to continue the iterative prototyping process.

During development, daily system updates are possible and allow for multiple tasks to work in parallel and to see how the tasks integrate. If something goes wrong, the impact does not affect the end user. However, it is more manageable during the maintenance stage of the system to have two-week releases. Also, as the system grows, the amount of testing done by the user increases, so the time line needs to be stretched out for them to manage his or her other responsibilities and also complete testing.

XP increases confidence between the customer and the developer. This aligns with other XP experts' opinion that the users do not have to trust the project team for a whole year to see the product for just two weeks [33]. Also, the level of commitment to the project is detected early. Building trust with the users was one way we diffused users' apprehension. By producing the product we said we would, when we said we would, paved the way for agility in the form of allowing developers to make processing decisions and creative designs and then ask the users for feedback. The users were busy, so the developers made some design decisions that in planned-driven methodologies, the user would have had to "sign off" on. The investment was relatively small: if the design decision was not what the user wanted, changes could be made and the amount of wasted effort was not that great.

This project's success was partially due to having a key stakeholder who was a strong champion for the project and embraced technology and change. Cockburn and Highsmith [18] identify several important users traits that are considered to be related to project success, including being committed, knowledgeable, collaborative, represented, and empowered. The evidence from the project suggests that this indeed made a difference in the success of this project.

Pair programming appears to work effectively and offer value to a project, but it is not always practically feasible. Especially projects that are in production and maintenance

mode that have a small team may not have the resources to double up for “pair programming.” There needs to be flexibility on the operationalization of this principle.

Not spending a lot of time on designing and developing the architecture for current and future requirements worked well on this project. As new technology became available, the additional architecture that was needed was developed. Only the architecture infrastructure the project needed for the current requirements was designed and developed, thus keeping the system simple, increasing development speed, and reducing overhead.

XP critics wonder how it can work to build systems that do not have all the requirements up front. Is there not a need to build extensive models of the problem and solution domains? And what about an extensive project plan that describes how the whole project will work [33]? It is speculated that if the project manager would have required a complete, comprehensive, large set of requirements, she would have scared the users away because of the hefty user up-front time commitment. The IT project team would not have gotten enough of the users’ time to do the requirements analysis in the plan-driven way and then the system would have never really gotten off the ground. The users did not have large chunks of time to devote to this project at the initiation of the project. Smaller intermittent chunks of time requirements helped the users to manage the time commitment needed for this project. This is one way the user overhead of the cost was manageable and minimized.

In most cases, either pure agility or pure plan-driven alone will not meet the needs. Combining agile and plan-driven methods is necessary [13, 18]. Selecting the “best practices” of the XP methodology and modifying a few of the practices was the best approach for this project. Previous research suggests that projects involving two to eight team members working in the same room with access to on-site users, one-month development increments, fully automated regression testing, and experienced developers are best suited for the agile methodology [21]. The outcomes of this study suggest that there is indeed room for flexibility on these requirements.

Conclusions

THE GOAL OF THIS RESEARCH WAS TO EXPLORE and better understand how to operationalize XP principles. This research concerned an emergency response system application for a state public health laboratory. It involved the development of a distributed, Web-based, secure application for laboratory technicians in hospital laboratories serving rural communities. This was a new and innovative system where no prior information system existed to guide the requirement identification. The XP approach was deemed appropriate for the following reasons: flexibility, rapid prototyping, low overhead, and it is methodology well suited for projects with fewer than ten team members.

As of this writing, 15 ERS have been implemented and are fully operational in various health labs. The transition of the system to a real work environment can be considered very successful. Fewer than ten updates had to be implemented due to errors; no updates had to be implemented due to wrong functionalities. The sponsor

of the ERS project is very satisfied as the labs are using the system. Additional funding has been made available to support the development of advanced functionalities and continue the rollout in additional health labs.

Overall, the study illustrates that several of the benefits of XP were realized, even though some of the XP principles had to be modified or extended and not all principles were implemented. The research found that where the technology is new or foreign, the XP process was flexible enough to support several iterations of technology and help the development team to produce prototypes in a timely manner. Some XP proponents argue that a project has to be fully adopted by all XP practices as they depend on each other [47]. On the other hand, many agree that no methodology is adopted 100 percent, there is always some adjustment [13]. When not allowing for some flexibility, innovation is stifled, and team effectiveness and efficiency may not be maximized. Furthermore, relaxing the requirement to adhere to 100 percent of the principles may represent one way for mission-critical systems and large complex systems to benefit from using the “best practices” of XP.

The main limitations of this study are twofold. First, this study concerns the in-depth analysis of a single XP project. One can wonder to what extent our findings can be attributed to the nature of the project itself and the people involved, or to the nature of the XP approach. Notwithstanding this concern, we feel that the purpose of our study is to provide an in-depth account of the experiences with XP in a real project with real users and a real system developed by real developers. The insights from this study are meant for future adopters of the XP approach to make informed decisions about the operationalization of the XP principles for their own situation. Second, more specifically, the project team consisted mostly of part-time developers. In industry, it is more common to utilize full-time developers. At the same time, it is not uncommon for developers to work on more than one project team at a time, especially for small to medium-sized projects. We therefore feel that our study has sufficient representative strength to make a valuable contribution to the body of knowledge on the application of XP principles in software engineering practice.

The results of this study suggest a number of avenues for future research. The first avenue concerns management aspects. Because of the fluid nature of defining and planning requirements, iterations, and prototypes in XP, budgeting is a challenge that we feel requires further investigation. Also, deeper understanding is needed regarding methods and techniques that managers can use to make investment decisions on these types of projects. The second avenue concerns the relative importance and success-critical nature of each of the XP principles. Which principles are most and which are least important with respect to the success of an XP project? Could an XP project be successful if one or several principles were absent? Comparative field studies could shed some light on these questions. A third avenue concerns project-related issues. So far, XP is mostly recommended for smaller project teams. The key question in this respect is, what are, if any, the best practices for the application of XP in larger project teams? Also, further research is required on how XP works in dualistic projects involving both development and production support. The final avenue for future research concerns the issue of how much documentation is necessary to ensure a smooth

transition when there is developer turnover or the system is “purchased” by another business needs to be studied.

Acknowledgments: The authors greatly appreciate the comments and suggestions from the anonymous reviewers that helped to strengthen the paper. They are indebted to Steven Hinrichs, Anthony Sambol, and Steven Stock for their ongoing enthusiasm and support during the ERS project.

REFERENCES

1. Abrahamsson, P., and Koskel, J. eXtreme programming: A survey of empirical data from a controlled case study. In N. Juristo and F. Shull (eds.), *Proceedings of the ACM–IEEE International Symposium on Empirical Software Engineering*. Los Alamitos, CA: IEEE Computer Society Press, 2004, pp. 73–82.
2. Anderson, A.; Beattie, R.; and Beck, K.; Bryant, D.; DeArment, M.; Fowler, M.; Fronczak, M.; Garzaniti, R.; Gore, D.; Hacker, B.; Hendrickson, C.; Jeffries, R.; Joppie, D.; Kim, D.; Kowalsky, P.; Mueller, D.; Murasky, T.; Nutter, R.; Pantea, A.; and Thomas, D. Chrysler goes to “extremes.” *Distributed Computing*, 11 (October 1998), 24–28.
3. Argyris, C.; Putnam, R.; and MacLain Smith, D. *Action Science—Concepts, Methods and Skills for Research and Intervention*. San Francisco: Jossey-Bass, 1982.
4. Armitage, J. Are agile methods good for design? *Interaction*, 11, 1 (January–February 2004), 14–23.
5. Astels, D.; Miller, G.; and Novak, M. *A Practical Guide to eXtreme Programming*. Upper Saddle River, NJ: Prentice Hall, 2002.
6. Auer, K., and Miller, R. *eXtreme Programming Applied*. Boston: Addison-Wesley, 2002.
7. Baskerville, R.L. Investigating information systems with action research. *Communications of the AIS*, 2, 19 (1999), 1–32.
8. Beck, K. Embracing change with eXtreme programming. *IEEE Computer*, 32, 10 (1999), 70–77.
9. Beck, K. *eXtreme Programming Explained*. Boston: Addison-Wesley, 2000.
10. Beck, K.; Beedle, M.; van Bennekum, A.; Cockburn, A.; Cunningham, W.; Fowler, M.; Grenning, J.; Highsmith, J.; Hunt, A.; Jeffries, R.; Kern, J.; Marick, B.; Martin, R.C.; Mellor, S.; Schwaber, K.; Sutherland, J.; and Thomas, D. Manifesto for agile software development. Agile Alliance, Snowbird, UT, 2001 (available at www.agilemanifesto.org).
11. Biffi, S.; Aurum, A.; Boehm, B.; Erdogmus, H.; and Grünbacher P. (eds). *Value-Based Software Engineering*. Berlin: Springer-Verlag, 2005.
12. Boehm, B.W. A spiral model of software development and enhancement. *IEEE Computer*, 21, 5 (May 1988), 61–72.
13. Boehm, B.W. Get ready for agile methods with care. *IEEE Computer*, 35, 1 (2002), 64–69.
14. Boehm, B.W. Value-based software engineering. *Software Engineering Notes*, 28, 2 (2003), 1–12.
15. Cao, L.; Mohan, K.; Xu, P.; and Ramesh, B. How extreme does programming have to be? Adapting XP practices to large-scale projects. In R.H. Sprague (ed.), *Proceedings of the Thirty-Seventh Hawaii International Conference on System Sciences*. Los Alamitos: IEEE Computer Society Press, 2004 (available at csdl2.computer.org/comp/proceedings/hicss/2004/2056/03/205630083c.pdf).
16. Checkland, P.B. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons, 1981.
17. Cockburn, A. *Agile Software Development*. Reading, MA: Addison-Wesley, 2001.
18. Cockburn, A., and Highsmith, J. Agile software development: The people factor. *IEEE Computer*, 34, 11 (2001), 131–133.
19. Cockburn, A., and Williams, L. The costs and benefits of pair programming. In *Proceedings of eXtreme Programming and Flexible Processes in Software Engineering*. Cagliari, Italy, 2000, pp. 223–243.

20. Elssamadisy, A., and Schalliol, G. Recognizing and responding to bad smells in eXtreme programming. In J. Magee and M. Young (eds.), *Proceedings of the Twenty-Fourth International Conference on Software Engineering*. New York: ACM Press, 2002, pp. 617–622.
21. Glass, R. Matching methodology to problem domain. *Communications of the ACM*, 47, 5 (May 2004), 19–21.
22. Grünbacher, P.; Halling, M.; Biffi, S.; Kitapci, H.; and Boehm, B.W. Integrating collaborative processes and quality assurance techniques: Experiences from requirements negotiation. *Journal of Management Information Systems*, 20, 4 (Spring 2004), 9–29.
23. Hansson, C., and Winter, J. Agile processes for small public projects and PD outside of office hours. In P. Flensburg (ed.), *Proceedings of the Twenty-Seventh Information Systems Research Seminar in Scandinavia*. Göteborg: IRIS Association, 2004, pp. 1–17.
24. Hickey, A.M., and Davis, A.M. A unified model of requirements elicitation. *Journal of Management Information Systems*, 20, 4 (Spring 2004), 65–84.
25. Highsmith, J., and Cockburn, A. Agile software development: The business innovation. *Computer*, 34, 9 (September 2001), 120–122.
26. Iivari, J.; Hirschheim, R.; and Klein, H.K. A dynamic framework for classifying information systems development methodologies and approaches. *Journal of Management Information Systems*, 17, 3 (Winter 2000–2001), 179–218.
27. Jeffries, R. eXtreme testing. *Software Testing & Quality Engineering*, 1, 2 (March–April 1999), 23–26.
28. Karlstrom, D. Introducing eXtreme programming—An experience report. In *Proceedings of the Third International Conference on eXtreme Programming and Agile Processes in Software Engineering*. Sardinia, Italy, 2002, pp. 24–29.
29. Keefe, K., and Dick, M. Using eXtreme programming in a capstone project. In R. Lister and A. Young (eds.), *Proceedings of the Sixth Australian Computing Education Conference*. Darlinghurst, Australia: Australian Computer Society, 2004, pp. 151–161.
30. Keefer, G. Pair programming: An alternative to reviews and inspections? *Cutter IT Journal*, 18, 1 (2005), 14–19.
31. Lethbridge, T.C.; Singer, J.; and Forward, A. How software engineers use documentation: The state of the practice. *IEEE Software*, 20, 6 (2003), 35–39.
32. Lippert, M.; Beck-Pechau, P.; Breitling, H.; Koch, J.; Kornstadt, A.; Roock, S.; Schmolitsky, A.; Wolf, H.; and Zullighoven, H. Developing complex projects using XP with extensions. *IEEE Computer*, 36, 6 (2003), 67–71.
33. Martin, R. eXtreme programming development through dialog. *IEEE Software*, 17, 4 (2000), 12–13.
34. Martin, R. *Agile Software Development: Principles, Patterns, Practice*. Upper Saddle River, NJ: Pearson Education, 2003.
35. McMahon, J. Five lessons from transitioning to eXtreme programming. *Control Engineering*, 50, 3 (March 2003), 59–65.
36. Muller, M.M., and Padberg, F. On the economic evaluation of XP projects. In P. Inverardi (ed.), *Proceedings of the Ninth European Software Engineering Conference Held Jointly with the Tenth ACM SIGSOFT International Symposium on Foundations of Software Engineering*. New York: ACM Press, 2003, pp. 168–177.
37. Myers, M.D. Qualitative research in information systems. *ISWorld Net*, June 1997 (available at www.qual.auckland.ac.nz).
38. Padberg, F., and Muller, M.M. Analyzing the cost and benefit of pair programming. In B. Werner (ed.), *Proceedings of the Ninth International Software Metrics Symposium*. Los Alamitos, CA: IEEE Computer Society Press, 2003, pp. 166–179.
39. Poole, C.; Murphy, T.; Huisman, J.; and Higgins, A. Extreme maintenance. In G. Canfora and A. Amschler Andrews (eds.), *Proceedings of the Seventeenth IEEE International Conference on Software Maintenance*. Los Alamitos, CA: IEEE Computer Society Press, 2001, pp. 301–312.
40. Saleem, N. An empirical test of the contingency approach to user participation in information systems development. *Journal of Management Information Systems*, 13, 1 (Summer 1996), 145–166.
41. Schach, S.R. *An Introduction to Object-Oriented Systems Analysis and Design with UML and the Unified Process*. Boston: McGraw-Hill, 2004.

42. Schwaber, K., and Beedle, M. *Agile Software Development with Scrum*. Englewood Cliffs, NJ: Prentice Hall, 2001.
43. Shore, J. Continuous design. *IEEE Software*, 21, 1 (2004), 20–22.
44. Smith, S., and Stoecklin, S. What we can learn from eXtreme programming. *Journal of Computing in Small Colleges*, 17, 2 (2001), 144–151.
45. Sommerville, I. *Software Engineering*. Boston: Addison-Wesley, 2004.
46. Standish Group. The CHAOS report into project failure. West Yarmouth, MA, 1994 (available at standishgroup.com/visitor/PDFpages/chaos1994.pdf).
47. Stephens, M., and Rosenberg, D. The irony of eXtreme programming. *Dr. Dobbs Journal*, 29, 5 (2004), 44–47.
48. Turner, R., and Boehm, B.W. People factors in software management: Lessons from comparing agile and plan-driven methods. *CrossTalk, The Journal of Defense Software Engineering*, 10, 12 (2003), 4–8.
49. Turoff, M.; Chumer, M.; Hiltz, R.S.; Klashner, R.; Alles, M.; Vasarhelyi, M.; and Kogan, A. Assuring homeland security: Continuous monitoring, control and assurance of emergency preparedness. *Journal of Information Technology Theory and Application*, 6, 3 (2004), 1–24.
50. Watson, R.T.; Kelly, G.G.; Galliers, R.D.; and Brancheau, J.C. Key issues in information systems management: An international perspective. *Journal of Management Information Systems*, 13, 4 (Spring 1997), 91–115.
51. Zuber-Skerritt, O. *Action Research for Change and Development*. Aldershot, UK: Gower, 1991.